MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Report on laboratory work no. 4

Course: Operating Systems

Topic: The xv6 Shell from the Outside

Done by: st. gr. FAF-241, Ion Dulgheru

Checked by: _____

Chişinău – 2026

Table of Contents

PURPOSE OF THE WORK....3
1 FIXING SLEEP FOR THE CURRENT XV6....
1.1 The name of the system call....
1.2 The updated program and its test....
2 BASH VERSUS THE XV6 SHELL....
2.1 Method....
2.2 Comparison table....
2.3 Answers to the Observe questions....7
3 WHO EXPANDS THE ASTERISK....
3.1 The programs args and args_linux....
3.2 Results....
3.3 Answers to the Observe questions....
4 READING THE SHELL....
4.1 Structure of sh.c....
4.2 Answers to the Observe questions....
CONCLUSIONS.... 15
REFERENCES....16

PURPOSE OF THE WORK

The purpose of the work is to compare the shell of xv6, which is a single small program (`user/sh.c`), with Bash, and to establish by experiment what a shell must do and what Bash adds on top of it [1]. A small program that prints the arguments it receives is used to show which component expands the asterisk and removes the quotes. Before that, the program `sleep` from Lab 2 is checked against the current version of xv6.

The work was performed on a Linux laptop; xv6 was built from the repository of the project [2] and run in the QEMU emulator. The files `sleep.c`, `args.c`, `args_linux.c` and the `Makefile` are stored in the folder `lab4/` of the course repository.

1 FIXING SLEEP FOR THE CURRENT XV6

1.1 The name of the system call

In August 2025 the authors of xv6 renamed the system call `sleep()` to `pause()`, so a program written for the old name may no longer build. The name used by the local tree was determined with:

```
cd ~/xv6-riscv
grep -n "pause\|sleep" user/user.h
```

The result is a line that ends with `int pause(int);` (line 25 of the tree used), which shows that the tree declares `pause` and does not declare `sleep` (figure 1.1). The program `sleep` written in Lab 2 already called `pause`, so no change of the source was required.

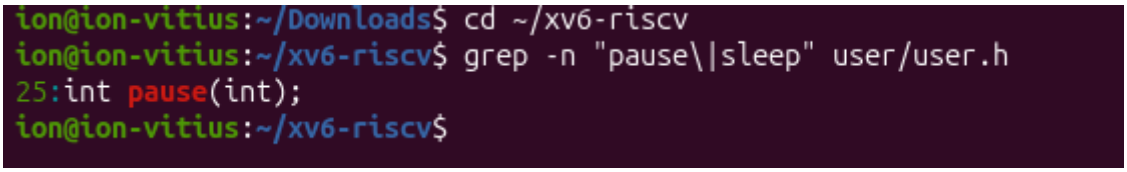


Figure 1.1 – The grep result in user.h

1.2 The updated program and its test

The source of `user/sleep.c` is given below; the line `$U/_sleep\` is present in the list `UPROGS` of the Makefile, with a tab at the beginning and the backslash at the end, with nothing after the backslash.

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    if(argc != 2){
        fprintf(2, "usage: sleep <ticks>\n");
        exit(1);
    }
    pause(atoi(argv[1]));             // was sleep() before Aug 2025
    exit(0);
}
```

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    if(argc != 2){
        fprintf(2, "usage: sleep <ticks>\n");
        exit(1);
    }
    pause(atoi(argv[1]));            // was sleep() before Aug 2025
    exit(0);
}
ion@ion-vitius:~/xv6-riscv$
```

```
ion@ion-vitius:~/xv6-riscv$ grep -n "_sleep\|_args" Makefile || sed -i 's#^\t$U/_zombie\\#\t$U/_zombie\\\n\t$U/_sleep\
\\n\t$U/_args\\#' Makefile
grep -n -B1 -A2 "_sleep" Makefile
148-    $U/_zombie\
149:    $U/_sleep\
150-    $U/_args\
151-    $U/_logstress\
ion@ion-vitius:~/xv6-riscv$
```

Figure 1.2 – The updated sleep.c

The system was rebuilt and started with `make qemu`, and the program was run with and without an argument: `sleep 20` paused and returned to the prompt, and `sleep` without an argument printed `usage: sleep <ticks>` (figure 1.3).

```
hart 1 starting
hart 2 starting
init: starting sh
$ sleep 20
$ sleep
usage: sleep <ticks>
$
```

Figure 1.3 – sleep running in xv6, with and without an argument

2 BASH VERSUS THE XV6 SHELL

2.1 Method

Every command of table 2.1 was run twice: in Bash, in a test folder that contains one file named `README` (`mkdir ~/shtest && cd ~/shtest && touch README`), and at the `$` prompt of xv6 started with `make qemu`. In xv6 the PATH row was typed as separate lines (`mkdir d`, `cd d`, `ls ..`), because the xv6 shell executes `cd` only when it is the first word of the line. The outputs are given in the table; the screenshots are shown in figures 2.1 and 2.2.

```
$ QEMU: Terminated
ion@ion-vitius:~/xv6-riscv$ mkdir -p ~/shtest && cd ~/shtest && touch README
echo hello > f ; cat < f
ls | grep READ
(echo a; echo b) | wc
echo one ; echo two
echo bg &
ls *
echo $HOME
echo "a b"
mkdir d ; cd d ; ls ..
hello
README
      2       2       4
one
two
[1] 12134
bg
README  f
[1]+  Done                    echo bg
/home/ion
a b
README  d  f
ion@ion-vitius:~/shtest/d$
```

Figure 2.1 – The commands run in bash

```
hart 1 starting
hart 2 starting
init: starting sh
$ rm f
$ echo hello > f
$ cat < f
hello
$ ls | grep READ
README          2 2 2441
$ (echo a; echo b) | wc
2 2 4
$ echo one ; echo two
one
two
$ echo bg &
bg
$ ls *
ls: cannot open *
$ echo $HOME
$HOME
$ echo "a b"
exec echo failed
$ echo "a b"
"a b"
$
```

Figure 2.2 – The same commands run in xv6

2.2 Comparison table

Table 2.1 – Behaviour of bash and of the xv6 shell

Command	Tests	bash	xv6	Why
echo hello > f ; cat < f	redirection	hello	hello	Both shells implement < and >
ls | grep READ	pipe	README	README 2 2 2441	Both implement pipes; the xv6 ls also prints type, inode and size
(echo a; echo b) | wc	grouping, list	2 2 4	2 2 4	Both support parentheses and ;
echo one ; echo two	sequence	one two (on two lines)	one two (on two lines)	; runs the commands in order
echo bg &	background	[1] 12134 and bg (job number and PID of the run)	bg (the prompt may appear before or after it)	Both implement & through fork without wait
ls *	globbing	README f (the files of the folder)	ls: cannot open *	xv6 does not expand *, so ls receives the literal name *
echo $HOME	variables	path of the home directory	$HOME	xv6 has no variables
echo "a b"	quotes	a b	"a b" (with the quotes)	xv6 does not process quotes, the quote marks stay in the words
mkdir d ; cd d ; ls ..	PATH	README d f	exec ls failed (lines typed separately)	xv6 has no PATH: ls is searched in the current directory d
arrow up, Tab, Ctrl+R	history, completion	recalls, completes, searches	arrow up prints the characters ^[[A and exec fails; Tab inserts a tab character (exec ech failed)	xv6 reads a plain line; no history and no completion

2.3 Answers to the Observe questions

Observe 1. Both shells have redirection (`<`, `>`, `>>`), pipes (`|`), command lists (`;`), grouping with parentheses and background execution (`&`). Only Bash has pattern expansion (`*`, `?`, `[ ]`, `{ }`), variables and the substitution `$( )`, quote processing, a search path (`PATH`), command history, completion and reverse search, and the operators `&&` and `||` with exit codes, together with job control and scripting.

Observe 2. In xv6 the shell calls `exec(argv[0], argv)` with the typed name and does not search any list of directories, so the name `ls` is looked up in the current directory. After `cd d` the current directory is `d`, which contains no file `ls`, and the shell prints `exec ls failed`. The xv6 programs are stored in the root directory, so the program can be started from inside `d` with its absolute path, `/ls`, or `/ls ..` to list the parent directory; the command `/ls` printed the entries `.` and `..` of the directory `d`. The result is shown in figure 2.3.



Figure 2.3 – ls failing after cd d, and the working alternative

Observe 3. The feature missed most in daily work is command history with completion, because every command has to be typed in full again. The easiest feature to add is a search path: when `exec` fails, the shell could try the same name in the root directory (`/` followed by the name), which would be a few lines in `runcmd`. This is a personal choice and may be replaced by another justified one, for example the variables or the quotes.

3 WHO EXPANDS THE ASTERISK

3.1 The programs args and args_linux

The program `user/args.c` prints the number of arguments and each argument between square brackets; it was added to the list `UPROGS` with the line `$U/_args\`:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    printf("argc = %d\n", argc);
    for(int i = 0; i < argc; i++)
        printf("argv[%d] = [%s]\n", i, argv[i]);
    exit(0);
}
```

The same program for Linux, `args_linux.c`, includes `<stdio.h>` instead of the two xv6 headers and uses `return 0`; it was compiled with `gcc -o args args_linux.c`.

```
#include "kernel/types.h"
#include "user/user.h"


int
main(int argc, char *argv[])
{
    printf("argc = %d\n", argc);
    for(int i = 0; i < argc; i++)
        printf("argv[%d] = [%s]\n", i, argv[i]);
    exit(0);
}
150:    $U/_args\
ion@ion-vitius:~/xv6-riscv$
```

Figure 3.1 – args.c and the UPROGS line

3.2 Results

The same four lines were run in both shells (in Bash, as `./args` in a folder with some `.c` files):

```
args one              two         three
args *.c "x y"
args $HOME
args 'single quoted'
```

In xv6 the outputs were: for `args one two three`, `argc = 4` with the arguments `one`, `two` and `three`; for `args *.c "x y"`, `argc = 4` with `*.c`, `"x` and `y"`; for `args $HOME`, `argc = 2` with `$HOME`; for `args 'single quoted'`, `argc = 3` with `'single` and `quoted'`. In Bash the same commands gave `argc = 4` for the first line; for the second line `argc` is the number of `.c` files plus 2 (for example, with the files `a.c` and `b.c`: `a.c`, `b.c` and `x y`, `argc = 4`), which was the case in the test folder; the third line gave `argc = 2` with the path of the home directory; the fourth gave `argc = 2` with `single quoted` as one argument. The results are shown in figures 3.2 and 3.3.



Figure 3.2 – args output in xv6 (top to bottom: one two three; *.c "x y"; $HOME; 'single quoted')

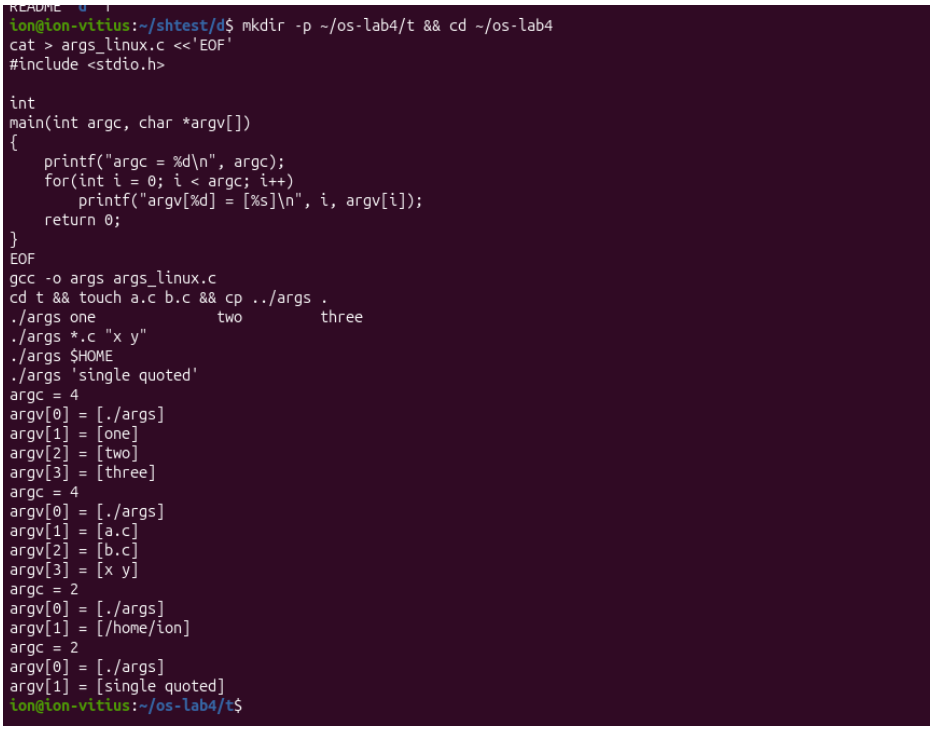


Figure 3.3 – args_linux output in bash

3.3 Answers to the Observe questions

Observe 1. For `args *.c "x y"`, the xv6 program received 4 arguments: `args`, `*.c`, `"x` and `y"`. The Linux program received the name of the program, the names of the `.c` files of the folder and the single argument `x y`: 4 arguments in the test folder (`./args`, `a.c`, `b.c` and `x y`).

Observe 2. The asterisk is expanded and the quotes are removed by the shell, not by the program and not by the kernel. The program only receives the finished list of arguments through `argv`; the kernel copies

this list into the new process during `exec`. The xv6 shell does neither of the two operations, which is why the program saw `*.c` and the quote marks unchanged.

Observe 3. The extra spaces in `one  two  three` disappeared: the shell treats whitespace only as a separator between words and discards it, so the program received exactly three arguments, `one`, `two` and `three`. Both shells behave in this way.

4 READING THE SHELL

4.1 Structure of sh.c

The following commands were run in `~/xv6-riscv`:

```
wc -l user/sh.c
grep -n "#define" user/sh.c
grep -n "case " user/sh.c
grep -n "cd" user/sh.c | head
```

The file has 499 lines. The command `grep -n "#define"` shows the five kinds of command (`EXEC`, `REDIR`, `PIPE`, `LIST` and `BACK`, lines 8 to 12) and the limit `MAXARGS 10` of arguments per command (figure 4.1). The command `grep -n "case "` shows where each kind is run, in the function `runcmd` (lines 75 to 125), and in the parser (figure 4.2). The search for `cd` finds only the error message of the built-in command, at line 170, so the code itself was read around the lines 166 to 170 (figure 4.3).



Figure 4.1 – The command kinds defined in sh.c



Figure 4.2 – Where each kind is run

```
  while (getcmd(buf, sizeof(buf)) >= 0) {
    char *cmd = buf;
    while (*cmd == ' ' || *cmd == '\t')
      cmd++;
    if (*cmd == '\n') // is a blank command
      continue;
    if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
      // Chdir must be called by the parent, not the child.
      cmd[strlen(cmd) - 1] = 0; // chop \n
      if (chdir(cmd + 3) < 0)
        fprintf(2, "cannot cd %s\n", cmd + 3);
    } else {
      if (fork1() == 0)
        runcmd(parsecmd(cmd));
      wait(0);
    }
ion@ion-vitius:~/xv6-riscv$
```

Figure 4.3 – The code that handles cd

4.2 Answers to the Observe questions

Observe 1. The shell knows five kinds of command, summarised in table 4.1.

Table 4.1 – Kinds of command of the xv6 shell

Kind	Character	Meaning
EXEC	none (a plain word)	A program with its arguments, started with exec
REDIR	< > >>	Redirection of the input or of the output of a command
PIPE	|	Two commands connected by a pipe
LIST	;	Two commands run one after the other
BACK	&	A command run in the background

Parentheses only group commands for the parser; they do not form a separate kind.

Observe 2. The command `cd` is handled in the main loop of the shell, in the lines 166 to 170 of `user/sh.c`: if the line starts with `cd `, the shell itself calls `chdir`, and the comment in the code states that `chdir` must be called by the parent, not by the child. It is not run as a separate program for the same reason as in Bash: the current directory belongs to the process, so a child process would change its own directory and terminate, and the shell would stay in the old one.

Observe 3. The search for the characters `*` and `$` finds no code that handles them: the symbols recognised by the parser are only `<`, `|`, `>`, `&`, `;`, `(` and `)`, and whitespace separates words. This explains the results of part 2: `ls *` and `echo $HOME` pass the text unchanged to the programs, so xv6 prints `ls: cannot open *` and `$HOME`. For the same reason, the line `mkdir d ; cd d ; ls ..` fails in xv6 with `exec cd failed`: only a line that starts with `cd` is handled by the shell itself, while inside a list the word `cd` is looked up as a program.

CONCLUSIONS

The work confirmed that the user interface of xv6 is a small program of about 500 lines that implements the essential functions of a shell: simple commands, redirection, pipes, lists, grouping, background execution and the built-in `cd` [1]. Everything else that makes Bash convenient, such as pattern expansion, variables, quotes, a search path and history, is an addition, and the experiments with `ls *`, `echo $HOME`, `echo "a b"` and `ls` after `cd d` showed what is missing in each case.

The program `args` established that the expansion of the asterisk and the removal of the quotes are performed by the shell before `exec`, and that the program and the kernel only receive and pass on the finished list of arguments. The program `sleep` from Lab 2 was checked against the current xv6, in which the system call is named `pause` [2].

REFERENCES

[1] Cox R., Kaashoek F., Morris R. xv6: a simple, Unix-like teaching operating system [online]. MIT. Available: https://pdos.csail.mit.edu/6.828/2023/xv6/book-riscv-rev3.pdf [accessed 05.10.2026].

[2] MIT PDOS. xv6-riscv source repository [online]. Available: https://github.com/mit-pdos/xv6-riscv [accessed 05.10.2026].